

Nazwa kwalifikacji: **Projektowanie, programowanie i testowanie aplikacji**
Oznaczenie kwalifikacji: **INF.04**
Numer zadania: **01**

Wypełnia zdający

Numer Identyfikacyjny zdającego*

--	--	--	--	--	--	--	--

Miejsce na numer ośrodka

Czas trwania egzaminu: **180** minut.

INF.04-KG-26.01

EGZAMIN ZAWODOWY

Rok 2026

CZĘŚĆ PRAKTYCZNA

**PODSTAWA PROGRAMOWA
2019**

Instrukcja dla zdającego

1. Na pierwszej stronie arkusza egzaminacyjnego wpisz w oznaczonym miejscu swój numer Identyfikacyjny.
2. Na KARCIE OCENY wpisz swoje dane:
 - swój numer Identyfikacyjny,
 - oznaczenie kwalifikacji,
 - numer zadania,
 - numer stanowiska.
3. Sprawdź, czy arkusz egzaminacyjny zawiera 7 stron i nie zawiera błędów. Ewentualny brak stron lub inne usterki zgłoś przez podniesienie ręki przewodniczącemu zespołu nadzorującego.
4. Zapoznaj się z treścią zadania oraz stanowiskiem egzaminacyjnym. Masz na to 10 minut. Czas ten nie jest wliczany do czasu trwania egzaminu.
5. Czas rozpoczęcia i zakończenia pracy zapisze w widocznym miejscu przewodniczący zespołu nadzorującego.
6. Wykonaj samodzielnie zadanie egzaminacyjne. Przestrzegaj zasad bezpieczeństwa i organizacji pracy.
7. Po zakończeniu wykonania zadania pozostaw arkusz egzaminacyjny z rezultatami oraz KARTĘ OCENY na swoim stanowisku lub w miejscu wskazanym przez przewodniczącego zespołu nadzorującego.
8. Po uzyskaniu zgody zespołu nadzorującego możesz opuścić salę/miejsce przeprowadzania egzaminu.

Powodzenia!

* w przypadku braku numeru ID – seria i numer paszportu lub innego dokumentu potwierdzającego tożsamość

Zadanie egzaminacyjne

UWAGA: katalog z rezultatami pracy oraz płytę należy opisać numerem zdającego, którym został podpisany arkusz. Dalej w zadaniu numer ten jest nazwany numerem zdającego.

Wykonaj aplikację konsolową oraz desktopową według wskazań. Udokumentuj obie aplikacje zrzutami ekranu. Wykorzystaj konto **Egzamin** bez hasła.

Utwórz folder i nazwij go numerem zdającego. W folderze utwórz podfoldery: *konsola*, *desktop*, *testy*. Po wykonaniu każdej aplikacji, jej pełny kod (cały folder projektu) spakuj do archiwum. Następnie pozostaw w podfolderze jedynie pliki źródłowe, których treść była modyfikowana, plik wykonywalny, jeśli jest to możliwe oraz spakowane archiwum.

Część I. Aplikacja konsolowa

Za pomocą narzędzi do tworzenia aplikacji konsolowych zaimplementuj program służący do zarządzania listą zwierząt przechowywaną w pliku tekstowym. Do zbudowania aplikacji wykorzystaj dane zwierząt (*dane.txt*) z archiwum **pliki1.7z** zabezpieczonego hasłem: **C3ntrum@dopcyjn3**

Założenia programu

- Program wykonywany w konsoli.
- Język programowania zgodny z zainstalowanym na stanowisku egzaminacyjnym, jeden z: C++, C#, Java, Python.
- Podejście obiektowe lub strukturalne.
- Program zawiera odczyt, zapis, dodawanie, sprawdzanie, szukanie danych.
- Program powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania kodu, należy stosować znaczące nazwy zmiennych i funkcji.
- Do kodu należy dołączyć testy jednostkowe opisane w części III zadania egzaminacyjnego.

Opis programu

Program składa się z funkcji/metody, typu wyliczeniowego i struktury. Użytkownik ma mieć możliwość odczytu danych z danych z pliku jak i ich zapisu, także sprawdzania czy numer jest zajęty, uzyskiwania danych po numerze, rodzaju i imieniu. Odczyt danych odbywa się z pliku który zawiera dane w wierszach oddzielone średnikiem: <numer>;<imie>;<rodzaj – nazwa>;<plec>

Założenia dotyczące kodu

- Program zawiera typ wyliczeniowy o nazwie `Rodzaj_zwierzecia` który zawiera wartości: kot, pies, kaczka
- Program zawiera struktury o nazwie `Zwierze` która zawiera pola:
 - A) numer (liczba całkowita)
 - B) imie (napis)
 - C) rodzaj (typ wyliczeniowy `Rodzaj_zwierzecia`)
 - D) plec (liczba całkowita, gdzie np. 0 – samica, 1 – samiec)
- Klasę która posiada:
 - A) prywatne pole klasy przechowujące obiekty typu `Zwierze`
 - B) Dwa konstruktory: 1 bezparametrowy i 2 przyjmujący jako argument ścieżkę do pliku z danymi aby wczytać dane
 - C) metody klasy:
 - 1) Metoda wczytująca dane zwierząt z pliku tekstowego zgodnie z formatem
 - 2) Metoda zapisująca aktualną listę zwierząt do pliku tekstowego zgodnie z formatem
 - 3) Metoda dodająca nowe zwierzę do listy
 - 4) Metoda sprawdzająca, czy podany numer zwierzęcia jest już zajęty (zwraca wartość logiczną)

- 5) Metoda zwracająca listę wszystkich zwierząt
- 6) Metoda wyszukująca zwierzęta po imieniu
- 7) Metoda wyszukująca zwierzęta po rodzaju
- 8) Metoda wyszukująca jedno zwierzę na podstawie numeru

- Nie jest wymagana walidacja tekstu – zakłada się, że wprowadzono go bez błędów
- Do kodu należy dołączyć testy jednostkowe opisane w części III zadania egzaminacyjnego.

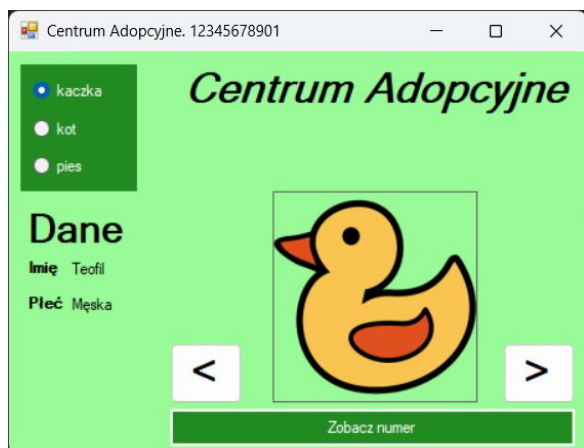
Sprawdź działanie metody lub funkcji sprawdzającej czy numer jest zajęty (wykorzystaj przykładowe dane). Program powinien nawiązać czytelną komunikację z użytkownikiem.

Po uruchomieniu programu wykonaj zrzuty ekranu dokumentujące interakcje programu dla różnych wartości numeru zwierzęcia. Zrzuty ekranu powinny obejmować cały obszar ekranu, z widocznym paskiem zadań oraz środowiskiem programistycznym. Liczba zrzutów ekranu powinna odpowiadać wszystkim możliwym interakcjom użytkownika z programem. Zrzuty ekranu zapisz w folderze konsolowa pod nazwami *konsola1.png*, *konsola2.png*, itd.

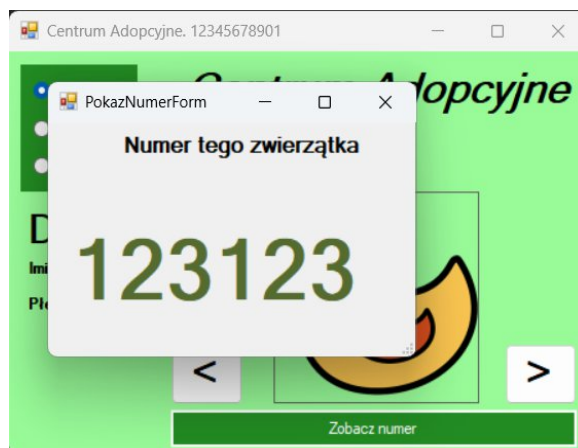
Kod aplikacji przygotuj do nagrania na płytę. W folderze konsolowa powinno znaleźć się archiwum całego projektu o nazwie *konsola.zip*, skopiowany z projektu plik z kodem źródłowym programu oraz plik wykonywalny, jeżeli istnieje.

Część II. Aplikacja desktopowa

Za pomocą środowiska programistycznego dostępnego na stanowisku egzaminacyjnym, wykonaj aplikację desktopową będącą graficznym interfejsem do zarządzania i przeglądania danych zwierząt wczytywanych z pliku tekstowego. Aplikacja desktopowa wykorzystuje kod utworzony w części I (aplikacja konsolowa).



Obraz 1. Aplikacja desktopowa, w stanie początkowym



Obraz 2. Aplikacja z wyświetlonym oknem numeru

Na obrazie 1 i 2 przedstawiono ideę aplikacji. W zależności od zastosowanego środowiska programistycznego wygląd może nieznacznie się różnić.

Opis wyglądu aplikacji

- Okno dialogowe o kolorze PaleGreen (#98FB98) i nazwie „Centrum Adopcyjne”. W nazwie okna należy wstawić także numerem zdającego.
- Okno zawiera kontrolki rozmieszczone zgodnie z obrazem 1. Są to:
 - Panel z trzema przyciskami typu RadioButton, umożliwiającymi wybór kategorii zwierząt: kaczka, kot, pies
 - Panel ma kolor ForestGreen (#228B22), a napisy RadioButtonów są w kolorze białym
 - Etykieta tytułowa z napisem „Centrum Adopcyjne”, wyświetlana dużą, pogrubioną czcionką
 - Obszar wyświetlania danych zwierzęcia:
 - etykieta „Imię” oraz pole wyświetlające imię zwierzęcia
 - etykieta „Płeć” oraz pole wyświetlające płeć zwierzęcia
 - Obszar wyświetlania obrazu:
 - kontrolka obrazu z obramowaniem
 - obraz dopasowany do rozmiaru kontrolki
 - wyświetlane obrazy: kaczka.png, kot.png, pies.png
 - Dwa przyciski nawigacyjne:
 - „<” – wyświetlenie poprzedniego zwierzęcia
 - „>” – wyświetlenie następnego zwierzęcia
 - Przycisk „Zobacz numer”, o zielonym tle takim samym jak panel i białej czcionce
- Drugie okno jest otwieranie w trybie modalnym.
- Okno zawiera kontrolki rozmieszczone zgodnie z obrazem 2:
 - Etykietę z napisem „Numer tego zwierzątka”
 - Numer identyfikacyjny zwierzęcia jest wyświetlany dużą, pogrubioną czcionką o kolorze DarkOliveGreen (#556B2F)

Opis działania aplikacji

- Po uruchomieniu aplikacji dane zwierząt są wczytywane z pliku tekstowego
- Domyślnie wyświetlana jest kategoria kaczka,
- Po wybraniu innej kategorii:
 - filtrowana jest lista zwierząt według wybranego rodzaju
 - wyświetlany jest odpowiedni obraz
 - ustawiana jest pierwsza pozycja z listy
- Przyciski „<” oraz „>” umożliwiają cykliczne przechodzenie pomiędzy zwierzętami z aktualnie wybranej kategorii
- Po wybraniu przycisku „Zobacz numer”:
 - otwierane jest drugie okno aplikacji
 - w oknie wyświetlany jest numer identyfikacyjny aktualnie wybranego zwierzęcia

Założenia aplikacji

- Komunikacja z użytkownikiem musi być czytelna
- Kod aplikacji powinien być zapisany czytelnie, z zachowaniem zasad czystego formatowania
- Należy stosować znaczące nazwy zmiennych, metod i kontrolek
- Grafiki wykorzystywane w aplikacji pochodzą z archiwum dołączonego do zadania egzaminacyjnego

Podejmij próbę kompilacji i uruchomienia aplikacji. Wykonaj zrzut ekranowy stanu początkowego aplikacji i stanu po wyświetleniu numeru zwierzęcia (drugie okno). Zrzuty ekranu zapisz w folderze desktopowa pod nazwami: *desktop1.png*, *desktop2.png*

Kod aplikacji przygotuj do nagrania na płytę. W podfolderze desktop powinno znaleźć się archiwum całego projektu o nazwie *desktop.zip* oraz plik (lub pliki) z kodem źródłowym modyfikowanym w czasie egzaminu.

Część III. Testowanie aplikacji konsolowej

Wykonaj testy jednostkowe wybranych metod klasy *Zwierze*, utworzonej w części I zadania (aplikacja konsolowa). Do napisania testów wykorzystaj dostępną na stanowisku egzaminacyjnym bibliotekę do testów jednostkowych. Testy jednostkowe powinny sprawdzać poprawność działania metod operujących na danych zwierząt wczytywanych z pliku tekstowego (dane z archiwum: *daneTestowe.txt*).

Założenia testów

- W testach wykorzystywany jest plik *daneTestowe.txt* zawierający dane zwierząt
- W każdej metodzie testującej należy użyć asercji
- Każdy przypadek testowy jest realizowany w osobnej metodzie.
- Nazwy metod testujących powinny jasno określać cel testu

Tabela 1. Scenariusz testowania z przypadkami testowymi

Nr testu	Cel testu (Jaka sytuacja będzie testowana)	Dane wejściowe	Oczekiwana wartość
1	Sprawdzenie, czy metoda poprawnie wykrywa zajęty numer zwierzęcia	numer = 123123	true
2	Sprawdzenie, czy metoda poprawnie wykrywa brak numeru w zbiorze	numer = 452123	false
3	Sprawdzenie poprawności wyszukiwania zwierzęcia po imieniu	imię = „Maria”	Obiekt <i>Zwierze</i> który ma imię Maria

Uruchomienie testów i dokumentacja:

- Po napisaniu metod testujących uruchom wszystkie testy jednostkowe
- Wykonaj zrzut dokumentujący wyniki wykonania metod testujących
- Zrzuty ekranu powinny obejmować cały obszar ekranu, z widocznym paskiem zadań oraz środowiskiem programistycznym
- Zrzuty zapisz w folderze testy pod nazwami: *test1.png*, *test2.png*, itd.

Utwórz archiwum ZIP zawierające projekt testowy (jeśli to potrzebne) i zapisz w folderze testy. W folderze z numerem zdającego utwórz plik pakietu biurowego o nazwie **egzamin** i umieść w nim:

- nazwę systemu operacyjnego, na którym pracował zdający,
- nazwy środowisk programistycznych, z których zdający korzystał na egzaminie,
- nazwy języków programowania użytych podczas tworzenia aplikacji
- opcjonalnie komentarz do swojej pracy

Tabela 2. Selected C# methods and features

Name	Description
Enum.Parse<TEnum>(string value)	Converts the string representation of the name or numeric value of an enumeration member to the equivalent enumeration object of type TEnum.
Any(predicate)	Determines whether any element of a collection satisfies a specified condition and returns a boolean value.
FindAll(predicate)	Searches a collection and returns a list of all elements that match the conditions defined by the specified predicate.
Find(predicate)	Searches a collection and returns the first element that matches the conditions defined by the specified predicate. If no element is found, a default value is returned.

UWAGA: Nagraj płytę z rezultatami pracy. W folderze z numerem zdającego powinny się znajdować podfoldery: konsola, desktop, testy i plik egzamin. W folderze konsola: cały projekt aplikacji konsolowej i zrzuty. W folderze desktop: cały projekt aplikacji desktopowej i zrzuty. W folderze testy: cały kod testów/archiwum aplikacji testującej, ewentualnie inne przygotowane pliki. Po nagraniu płyty sprawdź poprawność nagrania. Opisz płytę numerem zdającego i pozostaw na stanowisku, zapakowaną w pudełku wraz z arkuszem egzaminacyjnym.

Czas przeznaczony na wykonanie zadania wynosi 180 minut.

Ocenie będą podlegać 4 rezultaty:

- implementacja, kompilacja, uruchomienie programu,
- aplikacja konsolowa,
- aplikacja desktopowa,
- testy aplikacji